

PROJECT INTRODUCTION

University Operations AI Agent

A chat-based educational support agent on top of the existing University DB

Runs fully local & offline with Ollama • no API key required

Maryline Karam

Aseel Menhem

Hana Tfaily

Team & Work Division

Maryline Karam

Data Layer (Layer 6)

schema.sql · seed_data.sql · policies.json

Memory System (Layer 5)

short_term.py · working_memory.py

long_term.py (user preferences)

Docker / Deployment

docker-compose.yml · Dockerfile

■ Presents: Slides 3 & 4

Overview + Architecture

Aseel Menhem

LLM Core (Layer 3)

app/llm/client.py · app/config.py

Ollama integration & provider routing

Workflow / Orchestration (Layer 2)

LangGraph: graph.py · nodes.py

router.py · state.py · prompts.py

■ Presents: Slides 5 & 6

Why Ollama + How to Run

Hana Tfaily

Tools Layer (Layer 4)

4 required + 5 bonus tools

db_helpers.py (shared lookups)

Streamlit UI (Layer 1)

streamlit_app.py · logging_system/

Evaluation Suite

tests/eval/ · test_cases.json

■ Presents: Slide 7

Tools & Live Demo

What This System Does

■ A conversational layer on top of the University Management System database

- Does not redesign the existing DB — adds a 4-layer agent on top

■ Students and staff chat with it to:

- Look up courses, sections, policies, and academic info
- Check enrollment eligibility before registering
- Request enrollment (with a required confirmation step)
- Generate student / institution / payroll reports

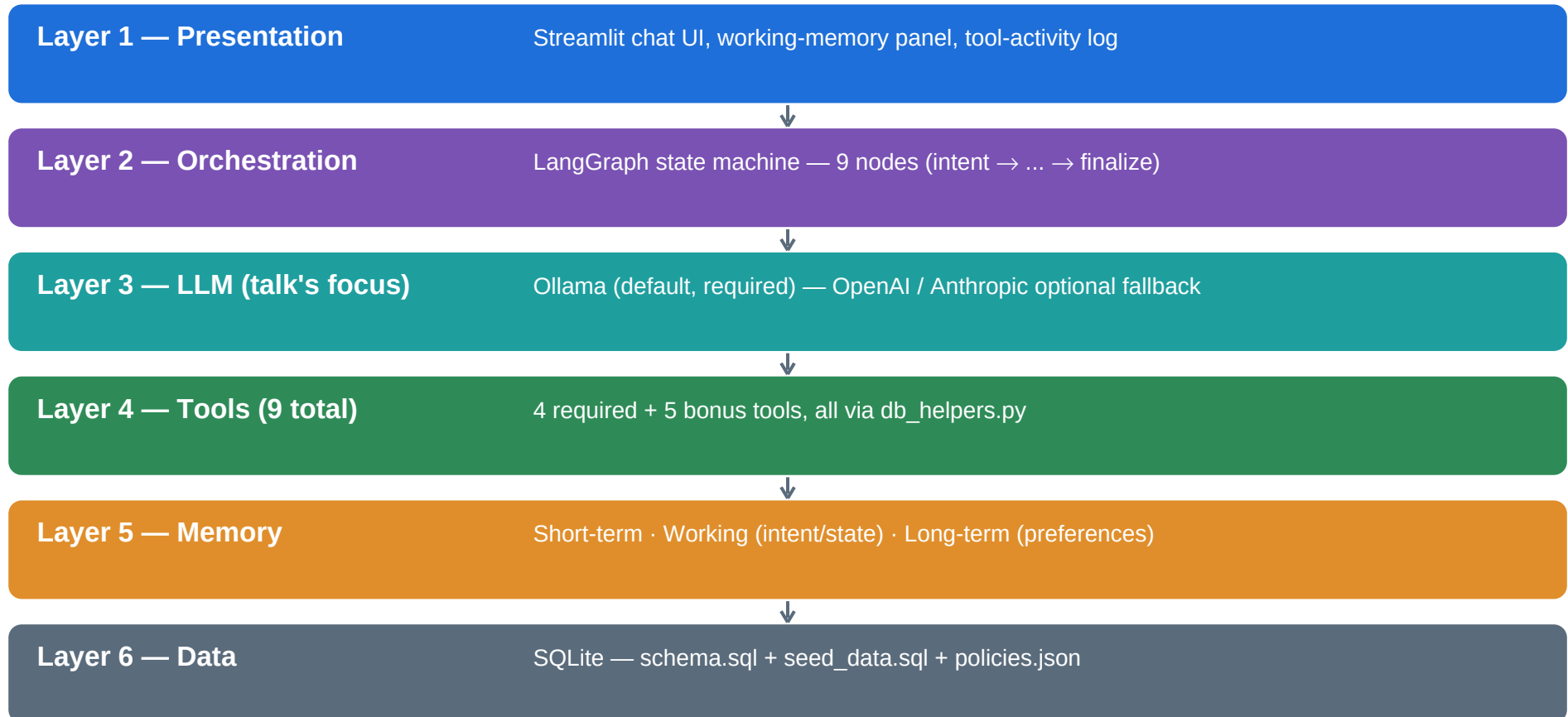
■ Built as an explicit, auditable LangGraph state machine

- Every turn: classify intent → gather info → validate → analyze → act → respond
- State-changing actions are gated behind a confirmation step

■ Configurable reasoning core

- The LLM powering intent classification is swappable via one environment variable

Architecture at a Glance



Why Ollama? Local-First by Design

■ Ollama is the default AND required LLM backend

- Model: llama3.1, served locally via OLLAMA_BASE_URL
- No API key, no account, no internet dependency once the model is pulled

■ OpenAI / Anthropic are optional fallback providers only

- Used only if LLM_PROVIDER + matching *_API_KEY are set in .env
- Never required, never a silent fallback — exactly one provider per run

■ One factory function controls everything

- app/llm/client.py → get_chat_model() reads LLM_PROVIDER, returns the right client
- Swapping providers is a config change — zero code changes anywhere else

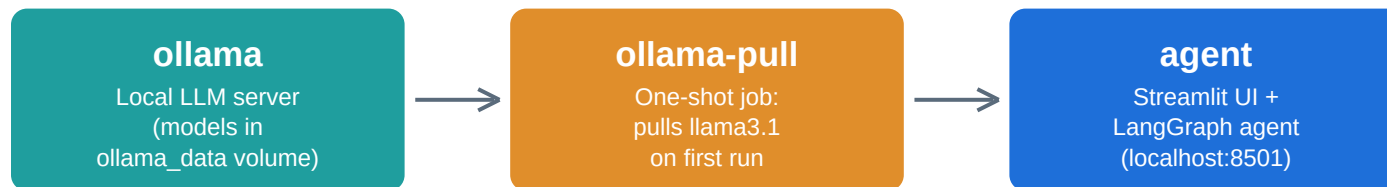
■ Fail-safe if the LLM is unreachable

- classify_intent() catches errors and returns intent="unsupported"
- Agent gives a safe refusal instead of crashing

Running It: One Command

```
$ cd ai_agent && docker compose up --build
```

No .env file. No API key. Then open <http://localhost:8501>



agent waits for ollama-pull to finish, then connects at <http://ollama:11434>

Without Docker:

1. `ollama serve`
2. `ollama pull llama3.1`
3. `pip install -r requirements.txt`
4. `python -m app.db.init_db`
5. `streamlit run streamlit_app.py`

Core Tools & Live Demo

Presented by
Hana Tfaily

4 Required Tools

- `get_university_information` — courses, policies, programs
- `analyze_enrollment_eligibility` — prereqs, GPA, capacity
- `create_enrollment_request` — confirmation-gated write
- `generate_student_report` — transcript / progress

5 Bonus Tools

- `predict_future_gpa`
- `generate_study_plan`
- `generate_payroll_report`
- `analyze_section_utilization`
- `generate_institution_report`

Live demo ideas:

- Ask about a course/policy
- Check eligibility
- Enroll → confirm → see it logged
- Generate a student report
- Show docker compose ps — all 3 services healthy